Following the Token: A Practical Guide to OAuth in PostgreSQL

PERCONA LIVE

Zsolt Parragi
Percona

Percona Live Amsterdam 2026



Who am I?



- Zsolt Parragi
- Software developer at Percona for 9 years
- Author of `pg_oidc_validator`
- Active contributor in PostgreSQL OAuth development

PERCONA LIVE

© 2026 Percona

What's OAuth/OIDC?

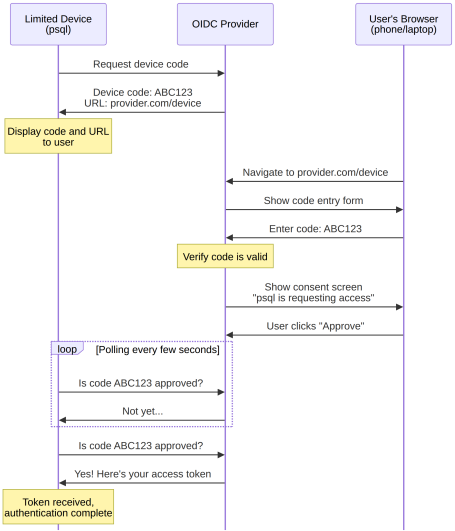


- Passwordless authentication
- Centralized identity management
- Seamless integration with enterprise identity providers
- Simple to use

PERCONA LIVE

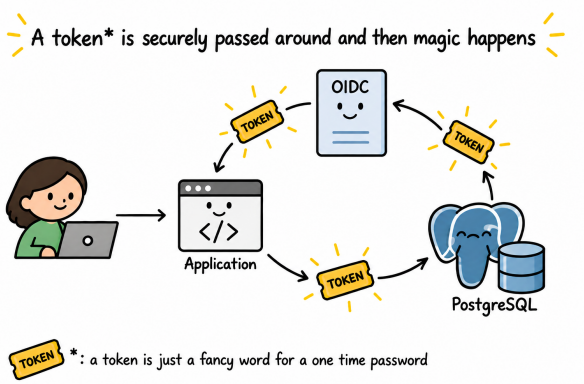
© 2026 Percona

▷ Simple?



© 2026 Percona

Even simpler?



© 2026 Percona

When something goes wrong



PERCONA LIVE

© 2026 Percona

▷OIDC requires extensions



- Token validation is not in core
- There are already multiple external validators
- There is at least one that doesn't properly validate the token

PERCONA LIVE

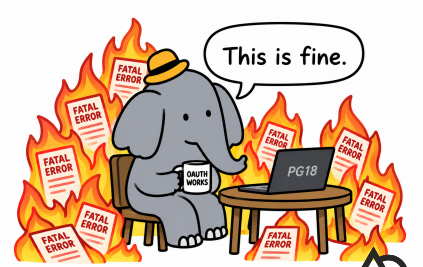
© 2026 Percona

A FATAL error is normal

- Fixed in PostgreSQL 19
- Log spam for every connection using OAuthBearer discovery:

```
FATAL:  OAuth bearer authentication
        failed for user "testuser"
```

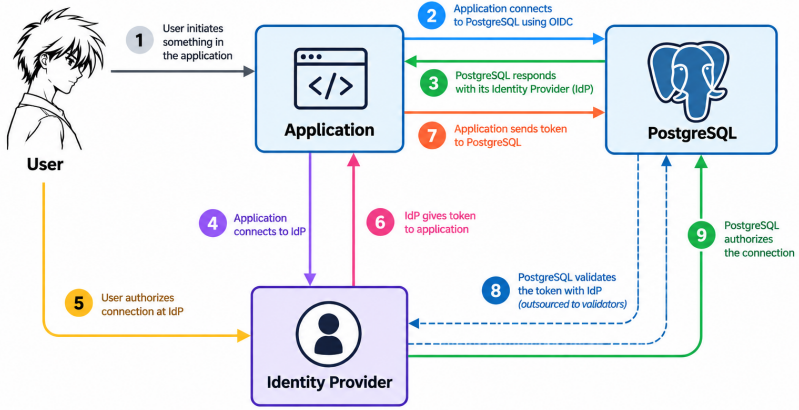
- Also displayed for real errors, but then there is a `WARNING` before it



PERCONA LIVE

© 2026 Percona

It’s just communication



© 2026 Percona

▷HBA issues

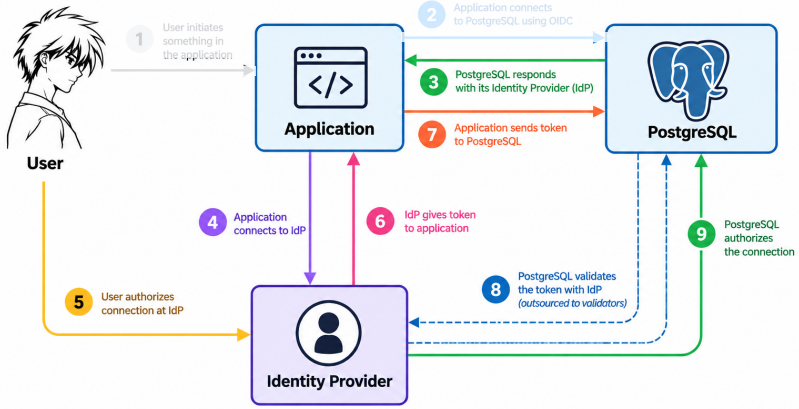


- `pg_hba.conf` uses the first matching (ip, username, database name) rule
- No easy migration path with a login method choice
- No support for multiple OIDC providers for the same database
- If the first rule matches ALL connections to OIDC, good luck!

PERCONA LIVE

© 2026 Percona

▷ It’s just communication



© 2026 Percona

Identity provider mismatch

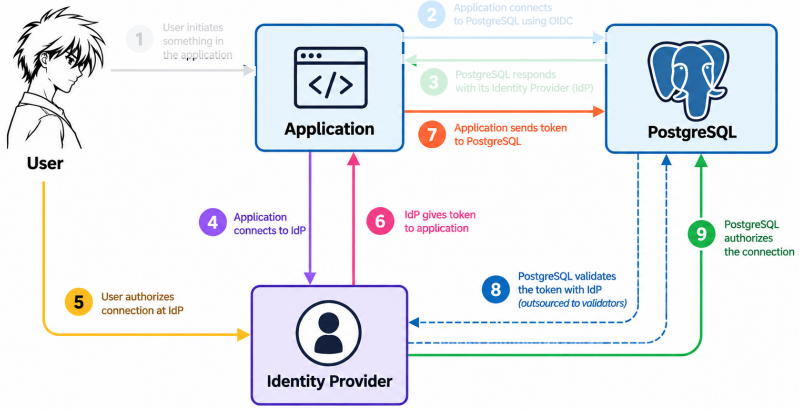


- Both the server and the client have to use the exact same canonical URL
- This mainly prevents malicious servers from extracting tokens
- Centralized authentication: you have to think about the server attacking the users!

PERCONA LIVE

© 2026 Percona

▷ It’s just communication



PERCONA LIVE

© 2026 Percona



User token verification

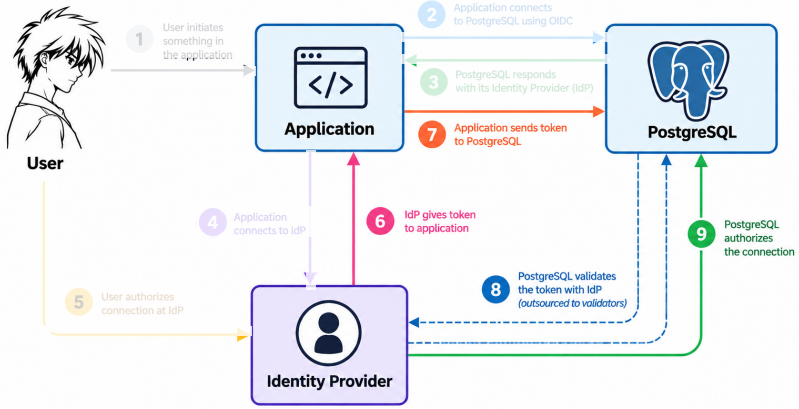
- A good[1] identity provider displays a detailed auth screen:
  - name of the client application
  - requested scopes
  - any additional details/warnings added by the client application
- Centralized authentication: you have to think about the server attacking the users!
- But also man in the middle and many other threats

[1] Not all identity providers are good

PERCONA LIVE

© 2026 Percona

▷ It’s just communication



PERCONA LIVE

© 2026 Percona

Authorization errors

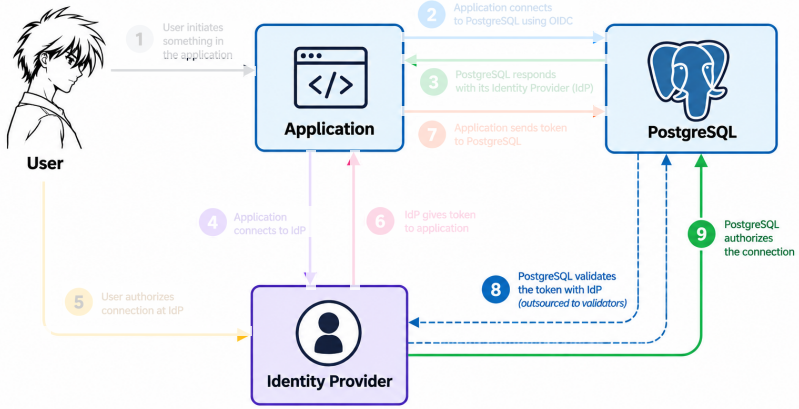


- Does the role returned by the validator exist in PostgreSQL?
- Does it have a proper `pg_ident.conf` mapping?
- Does the role pass the usual requirements of connecting to the database?

PERCONA LIVE

© 2026 Percona

▷ It’s just communication



© 2026 Percona

Validating the token



- Exact mechanism is up to the validator
  - a direct introspection endpoint
  - validating the cryptographic signature of the token
- Required claims/validity have to match

PERCONA LIVE

© 2026 Percona

Introspection endpoint



- The token is sent as-is to the IdP
- IdP answers if this is a valid token issued by it
- IdP also answers token details (permissions)
- Validator checks this against the PostgreSQL configuration
- Every backend has to connect to the IdP
- Token revocation is immediate

PERCONA LIVE

© 2026 Percona

JWKS signature

- Validator retrieves the public keys of the IdP
  - Usually[2] can be cached for a long time
- Validator verifies the signature against the public key
- No revocation: issued tokens remain valid
- Validator verifies token content:
  - issuer URL
  - scopes
  - expiry time
  - audience
  - other claims

[2]Except some providers

© 2026 Percona

Expired tokens



- OIDC has a concept of token refresh
- PostgreSQL doesn't know about refresh tokens
- Validity is only checked at connection time
- Applications/clients have to make sure to refresh tokens as needed

PERCONA LIVE

© 2026 Percona

Thank you!

